

# Product Description: Event-Driven Multi-Agent Platform MVP

## Product Description: Event-Driven Multi-Agent Platform MVP

### 1. Vision & Purpose

You are building a **distributed, event-driven multi-agent platform** that enables:

- Real-time agent execution
- Human-in-the-loop decision points
- Multi-agent workflows
- Dynamic agent schemas
- Frontend session-based interaction
- Polyglot agent support (Node + Python)

The MVP is designed to be **modular, scalable, and future-proof**, with a clear separation of concerns:

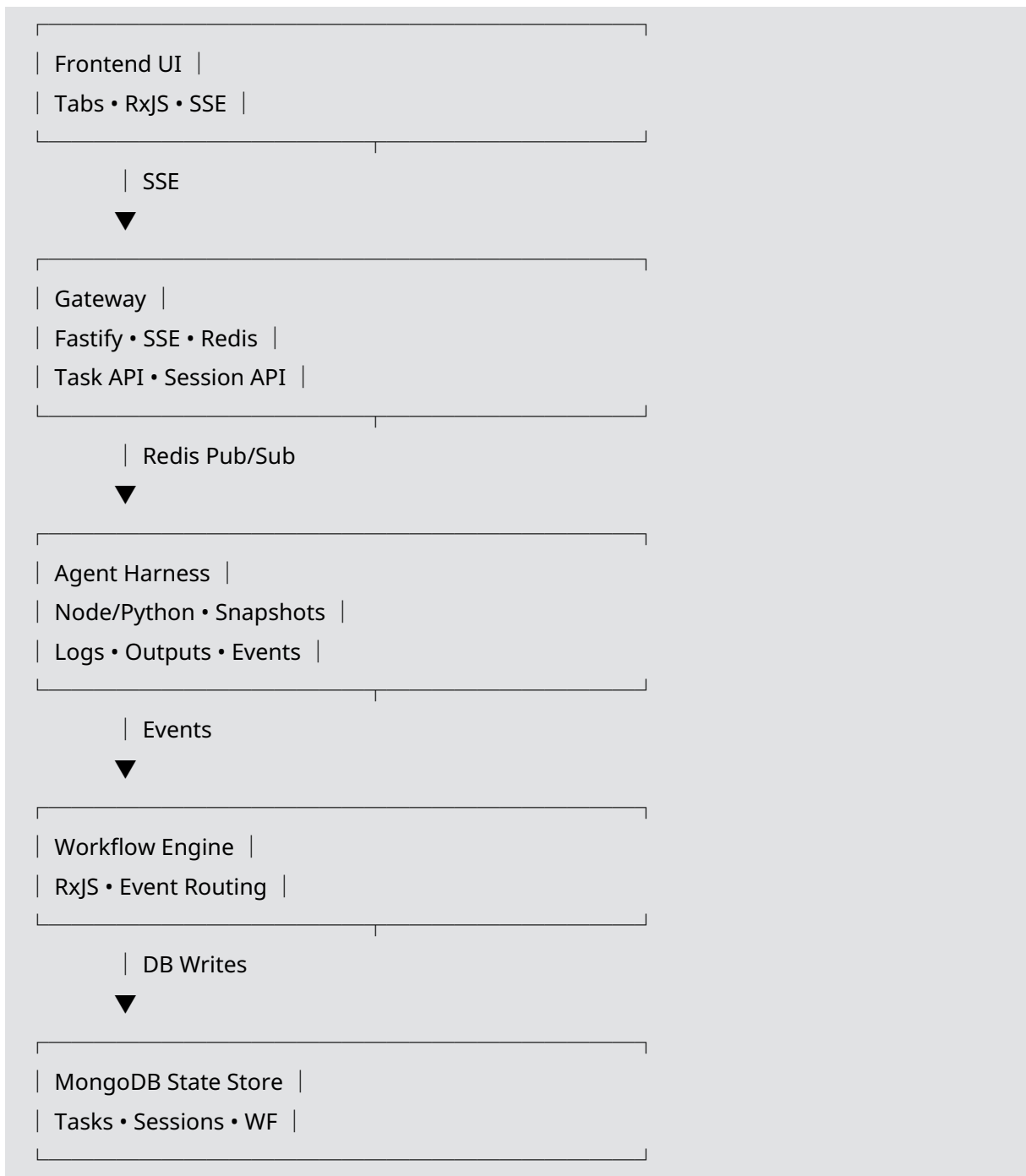
- **Gateway** orchestrates tasks, sessions, SSE, and Redis routing.
- **Agent Harness** executes agents, manages snapshots, and emits events.
- **Workflow Engine** reacts to events and drives orchestration.
- **Workflow Orchestration Layer** coordinates multi-agent pipelines.
- **MongoDB State Store** persists everything.
- **Frontend** binds tabs → sessions → SSE → RxJS → UI.
- **Agent Schema Loader** enables dynamic agent capabilities.

This architecture allows you to evolve from a simple agent runner into a full **workflow-driven agent ecosystem**.

---

### 2. High-Level Architecture

Below is the conceptual architecture showing how all components interact:



### 3. How Each Downloaded Document Fits Into the Architecture

#### Gateway Scaffold

Defines the Fastify server, SSE endpoints, Redis integration, and task/session routes.

#### Agent Harness Scaffold

Executes agents, handles pause/resume, publishes logs and outputs.

## Redis Event Schema

Standardizes all event shapes across Gateway, Harness, Workflow Engine, and Frontend.

## Task/Session State Store

MongoDB models for tasks, sessions, snapshots, workflow state.

## Workflow Persistence Layer

Stores workflow definitions, execution state, node/edge state, checkpoints.

## Workflow Orchestration

Runs workflows, triggers nodes, handles edge propagation, manages pause/resume.

## New Session Flow

Defines how sessions are created, bound to tabs, and connected via SSE.

## Session Event Fan-Out

Routes agent events to per-session SSE channels.

## Agent Schema Loader

Loads agent schemas, validates input, enables dynamic forms.

All documents together form a **complete, integrated MVP architecture**.

---

## 4. MVP Capabilities

### ✓ Real-time agent execution

Agents stream logs, partial output, and final output via SSE.

### ✓ Human-in-the-loop workflows

Agents pause, request approval, resume with user choice.

### ✓ Multi-agent workflows

Workflow orchestration triggers nodes and edges across agents.

### ✓ Dynamic agent schemas

Frontend renders forms based on agent schema.

### ✓ Session-based UI

Each tab is a session with its own SSE stream.

### ✓ **Polyglot agent support**

Node and Python agents run interchangeably.

### ✓ **Event-driven architecture**

Redis Pub/Sub connects all components.

### ✓ **Durable persistence**

MongoDB stores tasks, sessions, workflow state, snapshots.

---

## 5. Future Capabilities (Post-MVP)

### **workflow visualization UI**

Graphical node/edge visualization.

### **multi-agent pipeline builder**

Drag-and-drop workflow creation.

### **agent capability registry**

Discoverable agent marketplace.

### **workflow debugging tools**

Inspect node state, snapshots, event history.

### **autoscaling agent runners**

Horizontal scaling of agent harnesses.

### **distributed workflow execution**

Run workflows across multiple machines.

---

## 6. Brief Implementation Plan

### **Phase 1 - Core Infrastructure**

- Implement Gateway
- Implement Redis integration
- Implement MongoDB models

## Phase 2 - Agent Harness

- Implement Node/Python harness
- Implement snapshot + pause/resume

## Phase 3 - Frontend

- Implement tabs
- Implement SSE client
- Implement dynamic forms

## Phase 4 - Workflow Engine

- Implement RxJS event routing
- Implement orchestration layer

## Phase 5 - End-to-End Integration

- Connect Gateway ↔ Redis ↔ Harness ↔ Frontend
- Validate session creation → agent execution → workflow progression

## Phase 6 - MVP Polish

- Error handling
- Logging
- Basic UI improvements

---

## Summary

This product description defines:

- What you are building
- How the architecture works
- How each downloaded document fits together
- What the MVP can do
- What future capabilities it unlocks
- A clear implementation plan

Your system is now fully specified and ready for implementation sequencing.